UNIVERSIDAD PRIVADA DEL NORTE

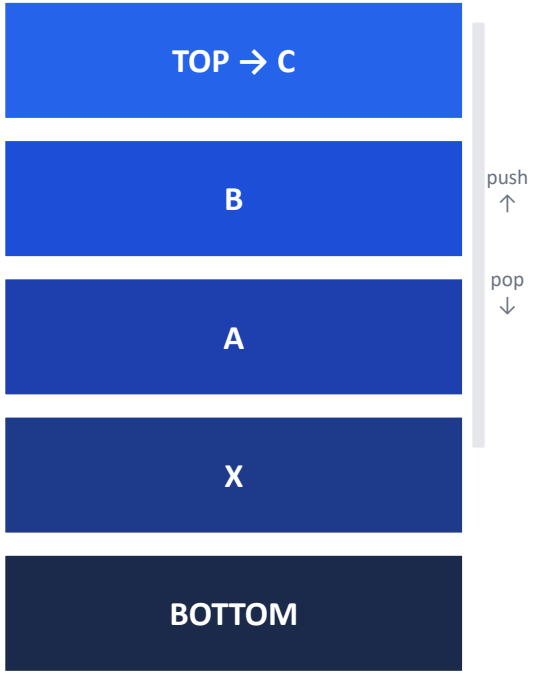
Facultad de Ingeniería • Ingeniería de Sistemas Computacionales

Curso: Estructura de Datos • Código: SIST1201A • Ciclo: 4° • Período: 2026-1

Pilas (Stacks)

Operaciones · Notación Prefija, Infija y Postfija
Análisis y Evaluación de Expresiones

SEMANA 5 • UNIDAD I



LIFO – Last In, First Out

Logro de la Sesión

Semana 5 · Unidad I

🎯 Logro

Al finalizar la sesión, el estudiante implementa la estructura de datos Pila en C#, aplica las notaciones prefija, infija y postfija, y evalúa expresiones aritméticas mediante algoritmos de conversión y evaluación, demostrando comprensión del principio LIFO.

📚 Conocimiento

Principio LIFO. Operaciones básicas de la Pila.

🔄 Habilidad

Convertir y evaluar expresiones en C#.

✅ Actitud

Buenas prácticas: código limpio y documentado.

Reflexión — Saberes Previos

¿Qué sabes sobre Pilas?

Antes de iniciar, reflexiona sobre las siguientes preguntas:

❓ **Pregunta 1**

Imagina que apilaste libros uno sobre otro y solo puedes retirar el del tope. ¿A qué estructura de datos te recuerda esta situación y por qué?

❓ **Pregunta 2**

Cuando evalúas la expresión matemática $3 + 4 \times 2$, ¿en qué orden deben ejecutarse las operaciones? ¿Cómo crees que una computadora sabe qué calcular primero?

💡 ***Discute con tu compañero antes de continuar.***

Reflexión — Respuestas

Verificación de saberes previos

✅ Respuesta 1 — La Pila (Stack)

Una pila es una estructura de datos lineal de tipo LIFO (Last In, First Out), donde el último elemento insertado es el primero en ser retirado. Esta propiedad es análoga a la pila de libros: solo se puede tomar el elemento del tope.

Fuente: Hernández Bejarano, M. & Baquero Rey, L. E. (2022). Estructuras de datos: fundamentación práctica. Editorial UOC.

✅ Respuesta 2 — Precedencia y Notación Postfija

Las computadoras utilizan la notación postfija (Reverse Polish Notation) para evaluar expresiones sin ambigüedad, eliminando la necesidad de paréntesis. En $3 + 4 \times 2$, la precedencia del operador $\times$ hace que se evalúe primero $4 \times 2 = 8$ y luego $3 + 8 = 11$. La pila es la estructura clave que permite realizar esta conversión y evaluación de forma eficiente.

Fuente: Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4.ª ed.). MIT Press.

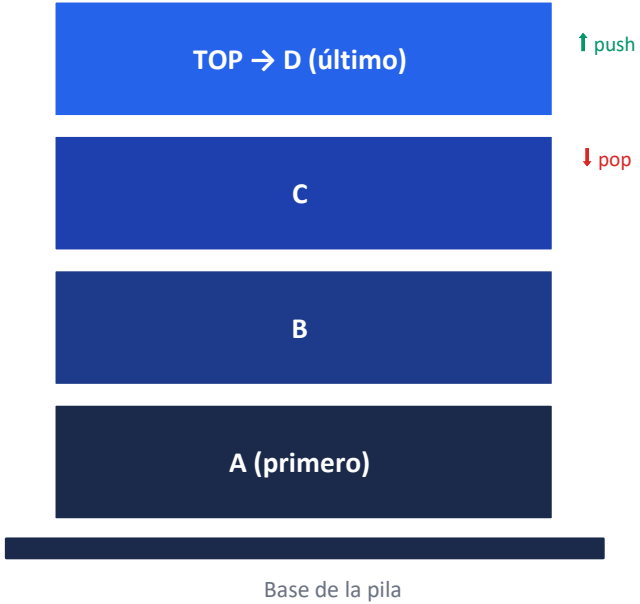
Pilas — Definición y Principio LIFO

Desarrollo · Parte 1

📖 Definición Formal

Una pila es una estructura de datos lineal en la que todas las inserciones y eliminaciones se realizan en un único extremo denominado tope (top). Su comportamiento sigue el principio LIFO: el último elemento en entrar es el primero en salir.

Hernández Bejarano & Baquero Rey (2022)



Analogías del mundo real

📚 Pila de libros

Solo se retira el del tope.

🍽️ Pila de platos

Último en apilarse, primero en usarse.

↩️ Deshacer (Ctrl+Z)

Revertir acciones en orden inverso.

🌐 Back del navegador

Retrocede a la última página visitada.

Fuente: Hernández Bejarano, M. & Baquero Rey, L. E. (2022). Estructuras de datos: fundamentación práctica. Editorial UOC.

Operaciones Fundamentales de la Pila

Desarrollo · Parte 2

⬆ Push(x)

Inserta el elemento x en el tope de la pila. Complejidad temporal: O(1).

Verifica si hay capacidad (pila estática) o asigna nodo (dinámica).

⬇ Pop()

Extrae y retorna el elemento del tope. Complejidad temporal: O(1).

Lanza excepción si la pila está vacía (underflow).

👁 Peek() / Top()

Retorna el elemento del tope sin eliminarlo. Complejidad: O(1).

Solo lectura. No modifica la estructura.

❓ IsEmpty()

Retorna true si la pila no contiene elementos.

Condición crítica antes de invocar Pop() o Peek().

🔢 Size()

Retorna el número de elementos en la pila.

Útil para depuración y control de capacidad.

⚡ **Todas las operaciones anteriores tienen complejidad O(1) — tiempo constante.**

Fuente: Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4.ª ed.). MIT Press. | Microsoft (2024). Stack<T> Class. https://learn.microsoft.com/dotnet/api/system.collections.generic.stack-1

Implementación en C# — Stack<T> Genérico

Desarrollo · Parte 3

```
// Implementación de una Pila genérica en C# (.NET 8)
// Fuente: Microsoft Learn — Stack<T> Class
// https://learn.microsoft.com/dotnet/api/system.collections.generic.stack-1

using System.Collections.Generic;

Stack<int> pila = new Stack<int>();

// Push: insertar elementos en el tope
pila.Push(10);   // Pila: [10]
pila.Push(20);   // Pila: [10, 20]
pila.Push(30);   // Pila: [10, 20, 30]  ← Tope

// Peek: consultar el tope sin eliminarlo
int tope = pila.Peek();      // tope = 30

// Pop: extraer el elemento del tope
int extraido = pila.Pop();   // extraido = 30 | Pila: [10, 20]

// IsEmpty: verificar si la pila está vacía
bool vacia = pila.Count == 0;  // false
```

💡 *Microsoft incluye Stack<T> en System.Collections.Generic desde .NET Framework 2.0. Documentación oficial: https://learn.microsoft.com/dotnet/api/system.collections.generic.stack-1*

Notaciones: Infija · Prefija · Postfija

Desarrollo · Parte 4

✏️ Infija (Infix)

El operador se ubica ENTRE los operandos. Es la notación habitual en matemáticas.

⚠️ *Requiere reglas de precedencia y paréntesis para desambiguar.*

Ejemplo:

A + B * C

⬅️ Prefija (Prefix / Polaca)

El operador se ubica ANTES de sus operandos. Propuesta por el lógico Jan Łukasiewicz.

⚠️ *No requiere paréntesis. Evalúa de derecha a izquierda.*

Ejemplo:

+ A * B C

➡️ Postfija (Postfix / Polaca Inversa)

El operador se ubica DESPUÉS de sus operandos. Óptima para evaluación con pilas.

⚠️ *Sin paréntesis ni precedencia. Evalúa de izquierda a derecha.*

Ejemplo:

A B C * +

Fuente: Cormen et al. (2022). Introduction to Algorithms, 4.º ed., MIT Press. | Knuth, D. E. (1997). The Art of Computer Programming, Vol. 1. Addison-Wesley.

Conversión Infija → Postfija: Algoritmo Shunting-Yard

Desarrollo · Parte 5

Algoritmo (Dijkstra, 1961):

1. Recorrer la expresión infija de izquierda a derecha.
2. Si es operando → añadirlo directamente a la salida.
3. Si es '(' → empujar a la pila.
4. Si es ')' → desapilar y enviar a salida hasta encontrar '('.
5. Si es operador → desapilar operadores de igual o mayor precedencia, luego empujar.
6. Al terminar → vaciar la pila hacia la salida.

Traza: Infija → A + B * C - D

Token	Operación	Pila	Salida
A	→ salida		A
+	push	+	A
B	→ salida	+	A B
*	push (*>+)	+ *	A B
C	→ salida	+ *	A B C
-	pop *, pop +	-	A B C * +
D	→ salida	-	A B C * + D
EOF	pop -		A B C * + D -

Resultado postfija: A B C * + D -

Fuente: Dijkstra, E. W. (1961). Shunting-yard algorithm. | Knuth, D. E. (1997). The Art of Computer Programming, Vol. 1. Addison-Wesley.

Evaluación de Expresión Postfija con Pila

Desarrollo · Parte 6

Algoritmo de evaluación postfija:

1. Recorrer la expresión de izquierda a derecha.
2. Si es operando → push a la pila.
3. Si es operador → pop dos operandos, aplicar la operación y push el resultado.
4. Al finalizar → el resultado es el único elemento en la pila.

Traza: 3 4 2 * +

Token	Acción	Pila
3	push 3	[3]
4	push 4	[3, 4]
2	push 2	[3, 4, 2]
*	pop 2,4 → 4×2=8 push 8	[3, 8]
+	pop 8,3 → 3+8=11 push 11	[11]
EOF	→ Resultado	11 ✅

✅ **Expresión: 3 + 4 × 2 = 11 | Postfija: 3 4 2 * + | Resultado: 11**

Fuente: Cormen et al. (2022). Introduction to Algorithms, 4.ª ed. MIT Press. Sección 10.1 — Stacks and Queues.

Código C# — Conversión Infija a Postfija

Desarrollo · Parte 7

```csharp
// Convierte expresión infija a postfija — Algoritmo Shunting-Yard
// Microsoft Learn: https://learn.microsoft.com/dotnet/csharp/
static string InfijaAPostfija(string expresion) {
    var salida = new StringBuilder();
    var pila  = new Stack<char>();
    var prec  = new Dictionary<char,int>{{'+',1},{'-',1},{'*',2},{'/',2}};

    foreach (char token in expresion.Replace(" ", "")) {
        if (char.IsLetterOrDigit(token)) {
            salida.Append(token + ' ');    // operando → salida
        } else if (token == '(') {
            pila.Push(token);
        } else if (token == ')') {
            while (pila.Peek() != '(') salida.Append(pila.Pop() + ' ');
            pila.Pop(); // descartar '('
        } else {   // operador
            while (pila.Count > 0 && pila.Peek() != '(' &&
                   prec.ContainsKey(pila.Peek()) && prec[pila.Peek()] >= prec[token])
                salida.Append(pila.Pop() + ' ');
            pila.Push(token);
        }
    }
    while (pila.Count > 0) salida.Append(pila.Pop() + ' ');
    return salida.ToString().Trim();
}
```

Fuente: Microsoft (2024). C# Programming Guide. https://learn.microsoft.com/dotnet/csharp/programming-guide/ | Dijkstra, E. W. (1961). Shunting-yard algorithm.

Código C# — Evaluación de Expresión Postfija

Desarrollo · Parte 8

```
// Evalúa una expresión en notación postfija usando Stack<double>
static double EvaluarPostfija(string postfija) {
    var pila = new Stack<double>();

    foreach (string token in postfija.Split(' ', StringSplitOptions.RemoveEmptyEntries)) {
        if (double.TryParse(token, out double num)) {
            pila.Push(num);   // operando → pila
        } else {
            double b = pila.Pop();  // segundo operando
            double a = pila.Pop();  // primer operando
            pila.Push(token switch {
                "+" => a + b,
                "-" => a - b,
                "*" => a * b,
                "/" => a / b,
                _   => throw new InvalidOperationException($"Operador inválido: {token}")
            });
        }
    }
    return pila.Pop();  // resultado final
}
```

💡 **Uso: `EvaluarPostfija("3 4 2 * +")` → retorna `11.0`**

Fuente: Microsoft (2024). Stack<T>.Pop Method. https://learn.microsoft.com/dotnet/api/system.collections.generic.stack-1.pop | Cormen et al. (2022). Introduction to Algorithms. MIT Press.

Actividad en Clase | Ejercicio 1 de 3

Conversión Infija → Postfija (manual)

Enunciado:

Convierte manualmente la siguiente expresión infija a su equivalente postfija, mostrando el estado de la pila en cada paso.

(A + B) * (C - D)

Completa la siguiente tabla de traza:

Token	Acción realizada	Estado de la Pila	Salida (Postfija)

Actividad en Clase | Ejercicio 2 de 3

Enunciado:

Evalúa la siguiente expresión postfija usando la Pila. Muestra el contenido de la pila en cada paso.

5 1 2 + 4 * + 3 -

Complete la tabla de evaluación y calcule el resultado:

Token	Acción	Contenido de la Pila	Resultado parcial

Resultado esperado: _____

Actividad en Clase | Ejercicio 3 de 3 — Programación C#

Enunciado — Reto de Programación:

Implementa en C# un método que verifique si una cadena de paréntesis está correctamente balanceada. Usa Stack<char>. Ejemplo: '({[]})' → balanceada; '({[)' → no balanceada.

```
// TODO: Implementa este método
static bool EstaBalanceada(string cadena) {
    var pila = new Stack<char>();
    // TODO: Definir los pares de apertura/cierre
    foreach (char c in cadena) {
        // TODO: Si es apertura → push
        // TODO: Si es cierre → verificar tope y pop
        // TODO: Si pila vacía o no coincide → return false
    }
    return ___________________; // ¿Cuándo está balanceada?
}

// Prueba: EstaBalanceada("({[]})") → true
//         EstaBalanceada("({[)")  → false
```

Pista: Usa un Dictionary<char,char> para mapear '{' → '}', '[' → ']', '(' → ')'.

Conclusiones

Semana 5 · Pilas: Operaciones y Notaciones

1. 📚 La Pila es LIFO

La pila garantiza que el último elemento insertado sea el primero en extraerse. Esta propiedad la hace ideal para gestión de llamadas, navegación y expresiones.

2. 🔄 Tres notaciones equivalentes

Infija (legible por humanos), prefija y postfija son formas equivalentes de representar expresiones. La notación postfija elimina la ambigüedad sin usar paréntesis.

3. ⚙️ El Algoritmo Shunting-Yard

Diseñado por Dijkstra (1961), permite convertir infija → postfija en O(n) respetando la precedencia de operadores mediante una pila auxiliar.

4. 💻 Stack<T> en C# (.NET)

La clase genérica Stack<T> de System.Collections.Generic implementa la pila con operaciones O(1): Push, Pop, Peek y Count.

🌟 **Las pilas son fundamentales en compiladores, sistemas operativos, editores de texto y evaluadores matemáticos.**

¿Preguntas?

¡Participa! Tu duda es la duda de muchos.

Referencias Bibliográficas

Formato APA 7.ª edición

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4.ª ed.). MIT Press.
   https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/

2. Hernández Bejarano, M., & Baquero Rey, L. E. (2022). Estructuras de datos: fundamentación práctica. Editorial UOC. Biblioteca UPN.
   https://elibro.bibliotecaupn.elogim.com/es/lc/upnorte/titulos/230581

3. Knuth, D. E. (1997). The Art of Computer Programming, Vol. 1: Fundamental Algorithms (3.ª ed.). Addison-Wesley.
   https://www-cs-faculty.stanford.edu/~knuth/taocp.html

4. Microsoft. (2024). Stack<T> Class. Microsoft Learn.
   https://learn.microsoft.com/dotnet/api/system.collections.generic.stack-1

5. Microsoft. (2024). Guía de programación en C#. Microsoft Learn.
   https://docs.microsoft.com/es-es/dotnet/csharp/programming-guide/

6. Sílabo SIST1201A. (2026). Estructura de Datos — Período 2026-1. Universidad Privada del Norte.
   Documento institucional UPN